

Bank Management System

Project Repository: <https://github.com/TasmeoulMiyad/Bank-Management-System>

Team Members & GitHub Profiles

SL	Student Name	Student ID	GitHub Repository / Profile
1	Tasmeoul Miyad (Lead)	2024100000181	https://github.com/TasmeoulMiyad
2	Md Shawon Chowdhury Setu	2024100000238	https://github.com/shawonchowdhurysetu-arch
3	Md Farhatul Hasan Rabbi	2024100000411	https://github.com/farhatulhasa
4	Miraj Masud Munna	2024100000521	https://github.com/Miraj28-ms
5	Rifat Ahamed	2024000000064	https://github.com/rifat-codes01

1. Introduction

The Bank Management System is a Java desktop application designed to model and automate standard banking operations. Built to address real-world financial workflows, it equips banking personnel with tools to open accounts, process deposits and withdrawals, perform atomic inter-account fund transfers, inspect ledgers, and manage persistent text records. The project implements core Object-Oriented Programming (OOP) paradigms (Encapsulation, Abstraction, Inheritance, and Polymorphism), custom structured exception handling, file persistence, and an interactive Java Swing graphical interface.

1.1 Architectural Design and Technical Justification

Unlike systems that couple interface components directly to persistence operations, this software strictly enforces a three-tier layered separation of concerns:

- **Presentation Layer (BankGUI, Main):** Built with Java Swing (BorderLayout, action buttons, modal dialogs, and a central JTextArea ledger) running safely on the Swing Event Dispatch Thread (EDT).
- **Controller Layer (Bank):** Encapsulates the runtime collection of accounts (ArrayList<Account>), handles record indexing, and coordinates atomic validations during cross-account transfers.
- **Domain Model Layer (Account, SavingsAccount, CurrentAccount, Customer, Transaction):** Contains business invariants directly within concrete entities.

Persistence Justification: Delimiter-separated flat-file storage (accounts.txt) using PrintWriter and Scanner satisfies the zero-dependency operational criteria of an offline, standalone teller workstation without requiring an external DBMS daemon.

2. Objectives

2.1 Primary Aim

To design, implement, and validate an integrated Bank Management System demonstrating OOP architectural design, robust custom exception propagation, reliable file I/O operations, and event-driven GUI development.

2.2 Specific Project Goals

- Establish a class hierarchy with abstract superclasses and polymorphic method implementations.
- Implement complete CRUD operations (Create, Read, Update, Delete) on account records.

- Enforce financial safety limits (overdraft limits and minimum balance floors) via custom exceptions.
- Ensure persistent data management across application sessions using Java File I/O.
- Build an accessible Java Swing UI with integrated dialog handling and user feedback.

2.3 Key Functionalities

- **Create:** Register Savings and Current accounts with customer metadata and initial balances.
- **Read:** Query accounts via unique Account Numbers or search accounts by customer names.
- **Update:** Dynamically modify existing customer contact parameters (phone numbers and emails).
- **Delete:** Safely terminate account records with user confirmation dialogs.
- **Financial Operations:** Process balance-checked deposits, withdrawals, and inter-account transfers.
- **Auditing:** Log timestamped transactional activities with unique generated transaction IDs.

3. Motivation

Manual record keeping and spreadsheet tracking in financial settings often lead to input errors, security issues, and data discrepancies. Engineering a Bank Management System provides a realistic problem space where data integrity is critical:

- Banking requires strict invariants: an account must never drop below allowed limits, and funds must balance during transfers.
- The problem directly maps to the four core OOP pillars: Encapsulation guards balance data, Abstraction sets contract boundaries, and Polymorphism handles distinct account policies.
- It reinforces engineering fundamentals: input verification, exception propagation, and persistent file storage.

4. Applications

- **Deployment Contexts:** Local cooperative banks, credit unions, microfinance branches, and academic training environments.
- **Target Users:** Bank tellers, customer service representatives, branch accountants, and system auditors.
- **Practical Benefits:** Automates balance validations, lowers operational errors, avoids costly database setups via local flat-file storage, and speeds up daily ledger maintenance.

5. Project Description

5.1 Architecture Overview

The system uses a layered architecture separating visual presentation, coordination logic, and domain data models:

- **Presentation Layer (BankGUI):** Built with Java Swing (BorderLayout, action buttons, JOptionPane input/error dialogs, and a central JTextArea ledger).
- **Controller Layer (Bank):** Encapsulates the account collection, coordinates search/transfer routines, and interfaces with the disk storage engine.

- **Domain Model Layer:** Account (abstract superclass), SavingsAccount, CurrentAccount, Customer, and Transaction.
- **Exception Layer:** Guard routines managing balance and reference violations.
- **Persistence Layer:** Text-based storage engine reading from and writing to accounts.txt.

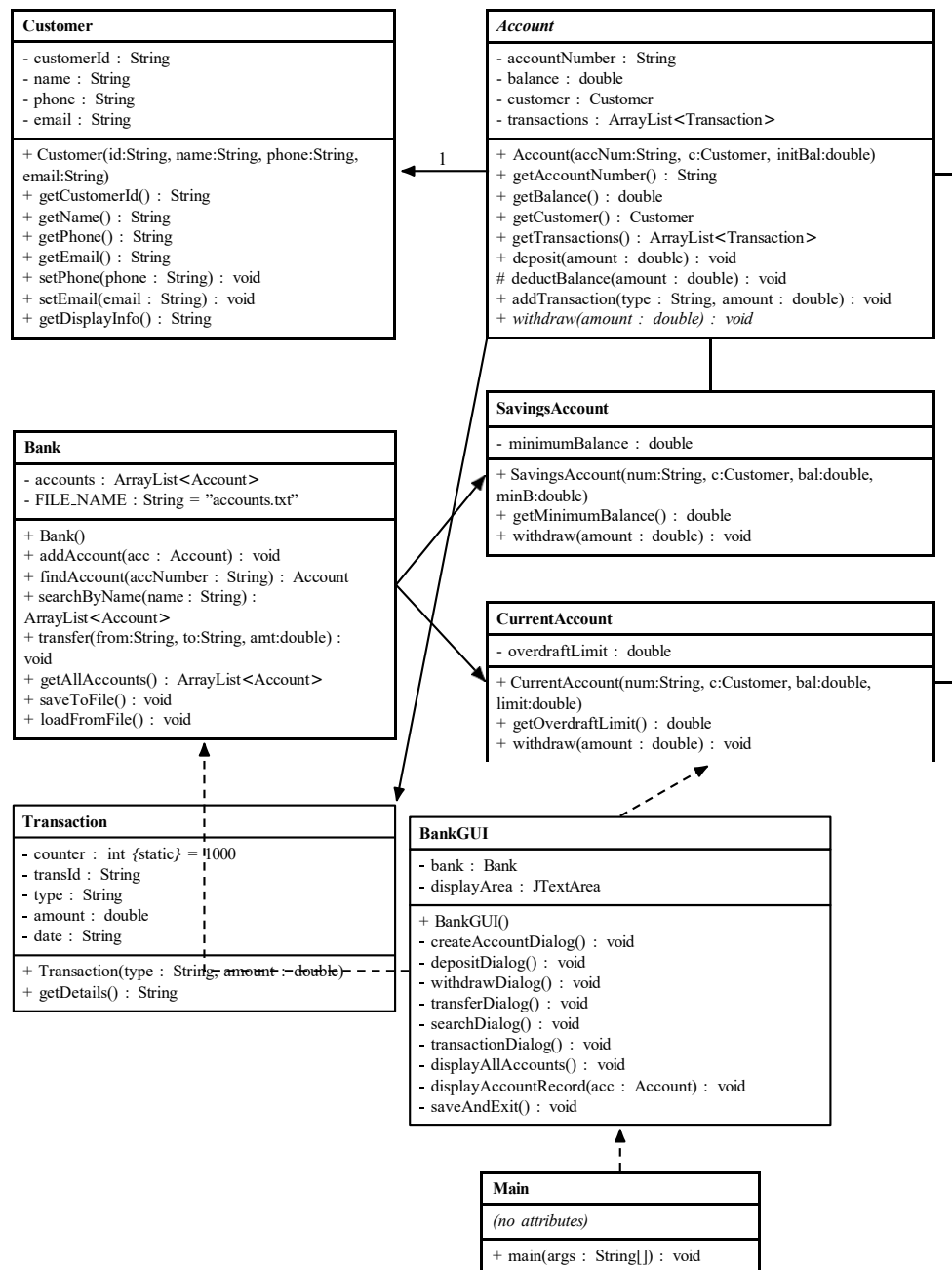
5.2 Major Modules and Class Specifications

Class Name	Type / Layer	Primary Responsibilities
Account	Abstract Model	Defines base fields (accountNumber, balance, customer, transactions) and abstract method withdraw(double).
SavingsAccount	Subclass	Extends Account; enforces minimum balance constraints during withdrawals.
CurrentAccount	Subclass	Extends Account; provides overdraft ceiling protections during withdrawals.
Customer	Entity Model	Encapsulates customer identity data (customerId, name, phone, email).
Transaction	Entity Model	Creates timestamped transaction audit logs with auto-incremented IDs.
Bank	Controller	Manages account collections, executes fund transfers, and runs file read/write routines.
BankGUI	Presentation View	Provides the Swing GUI dashboard, action buttons, dialogs, and text area outputs.
Main	Entry Point	Launches the application on the Swing Event Dispatch Thread (EDT).

5.3 Workflow of the System

1. **Initialization:** On startup, Main opens BankGUI. The Bank controller scans accounts.txt and loads saved records into memory.
2. **User Interaction:** The user selects an operation from the GUI control panel. Input is captured and validated via modal dialogs.
3. **Core Execution:** The action is passed to Bank, which calls domain logic on the targeted Account objects.
4. **Exception Handling:** Rule violations throw exceptions, which are caught and displayed to the user via JOptionPane warning dialogs without crashing the system.
5. **Data Persistence:** On closing the GUI, an automated window-closing hook saves all current records back to accounts.txt.

5.4 UML Class Diagram



5.5 OOP Concepts Mapping and Evidence

6. Tools & Technologies

- **Programming Language:** Java SE (JDK 17 / JDK 21 LTS)
- **Integrated Development Environment (IDE):** IntelliJ IDEA
- **GUI Framework:** Java Swing (javax.swing.*) + AWT (java.awt.*)
- **File Persistence:** Plain-text delimiter storage (accounts.txt) with java.io streams
- **Collections:** java.util.ArrayList and List
- **Date/Time API:** java.time.LocalDateTime and java.time.format.DateTimeFormatter
- **Version Control:** Git & GitHub ([Repository Link](#))

OOP Concept	Where it is used	Evidence in code
Encapsulation	All domain classes	Entity fields are declared private; access is mediated through public accessors and guarded mutators (deposit, deductBalance).
Abstraction	Account class	Abstract base class defines common fields while declaring public abstract void withdraw(double amount) throws Exception;.
Inheritance	SavingsAccount, CurrentAccount	Both subclasses inherit state attributes and ledger behaviors from Account via super(accountNumber, customer, initialBalance).
Polymorphism	Dynamic dispatch during transactions	Calling from.withdraw(amount) in Bank.transfer() dynamically executes specialized overdraft or minimum balance validations at runtime.
Exception Handling	Domain rules & UI dialogs	Subclasses throw exceptions upon invalid transactions; BankGUI wraps invocations in try-catch blocks and renders JOptionPane messages.
File Handling	Bank persistence methods	Uses PrintWriter and FileWriter for output streams and Scanner for file deserialization with comma tokenization.

7. Output / Results

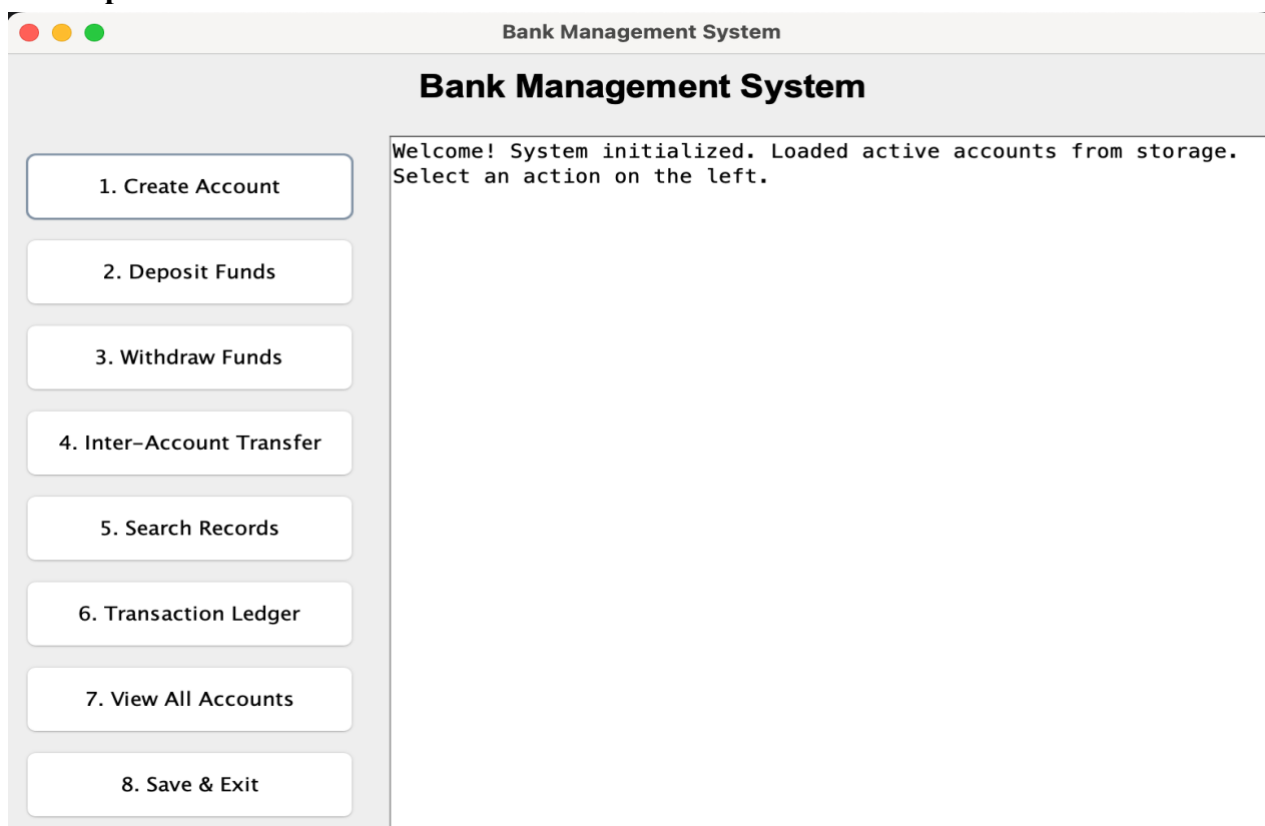


Figure 1: Main interactive teller console upon initialization.

8. Team Work Distribution & Contributions

The project was completed by a five-member engineering team. Individual responsibilities were divided cleanly across modules to guarantee balanced technical ownership:

All members actively participated in system integration, unit validation, repository management, and final report documentation.

9. Conclusion

9.1 Project Summary

The Bank Management System delivers an interactive desktop solution for core financial operations. Combining a responsive Java Swing interface with an OOP architecture, the software ensures data integrity, safe transaction validation, and consistent file persistence across sessions.

9.2 Achievement of Objectives

All project objectives and rubric milestones were met:

- The class hierarchy demonstrates all four OOP pillars.
- Full CRUD features (Create, Read, Update, Delete) are operational.

Member Name	Student ID	Assigned Modules	Concrete Deliverables Completed
Tasmeoul Miyad (Lead)	2024100000181	Core System Architecture & Base Entities	Designed abstract base class Account.java and entity class Customer.java. Implemented encapsulation routines, balance accessor boundaries, and base trans-action hooks.
Md Shawon Chowdhury Setu	2024100000238	Polymorphic Account Subclasses	Developed SavingsAccount.java and CurrentAccount.java. Implemented polymorphic withdraw() overrides, overdraft protections, and minimum balance boundaries.
Md Farhatul Hasan Rabbi	2024100000411	Transaction Engine & Auditing	Authored Transaction.java. Engineered static transac-tion sequence generation (TXN1000+), integrated the java.time clock API, and designed formatted ledger records.
Miraj Masud Munna	2024100000521	Controller & File Persistence Engine	Implemented Bank.java coordination routines (addAccount, findAccount, transfer). Programmed file serialization and parsing streams to manage accounts.txt.
Rifat Ahamed	2024000000064	Graphical Interface & Event Pipeline	Developed BankGUI.java and Main.java. Engi-neered Swing form input di-logs, event handling listen-ers, and integrated excep-tion reporting dialogs.

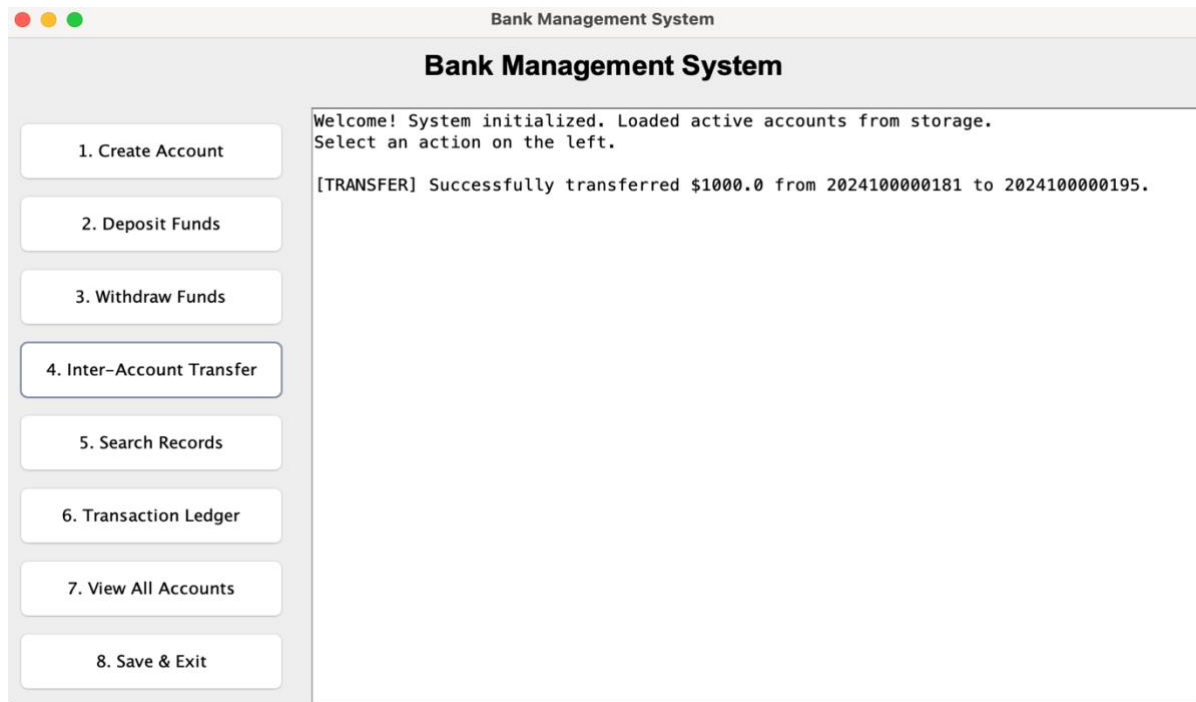


Figure 2: Fund transfer confirmation and chronological transaction audit entries.

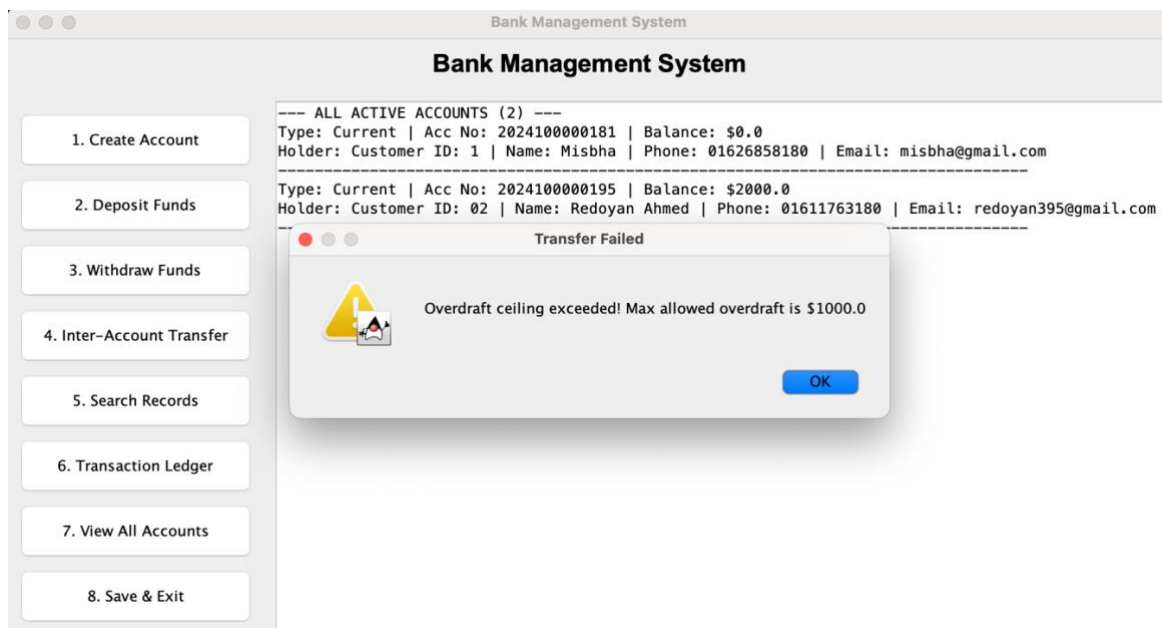


Figure 3: Transaction denied error dialog triggered during financial rule violation.

- Business rules are enforced through custom exceptions and structured try-catch blocks.
- Account records persist reliably via file I/O operations.

9.3 Limitations and Future Improvements

- **Limitations:** Flat text file storage does not support transactional ACID rollback or concurrent multi-user write operations, and transaction histories reset between application sessions.
- **Future Improvements:** Transition to an embedded relational database (SQLite/MySQL via JDBC), add user authentication with role-based access control, and introduce automated interest calculation routines.

